A Survey of Modern Go AI: Search, Network Architectures, Engines, and Robustness

June 2026

Abstract

We survey the modern era of computer Go, from the AlphaGo breakthrough through the self-play template that now underlies every strong engine. We organize the field along four axes: the *search and planning* algorithms that turn a network into a player (MCTS/PUCT, MuZero, Gumbel search); the *network architectures* that evaluate positions (residual trunks, KataGo's nested bottlenecks, and a recent wave of transformer backbones); the *engines and implementations* that competed in the tournament era (open reproductions, industrial systems, and academic engines); and the *adversarial-robustness* results showing that supposedly superhuman engines retain an exploitable blind spot. We close with emerging directions — language-model planning, learned world models, and interpretability — and with the open questions that follow from the cyclic-attack literature.

1 Introduction and scope

The game of Go was for decades the canonical hard problem for game AI: a 19×19 board, a branching factor in the hundreds, and a positional-evaluation problem that resisted the handcrafted heuristics that had cracked chess. Monte-Carlo Tree Search (MCTS) made the first dent in the 2000s — engines such as **Crazy Stone** and **Zen** reached strong-amateur strength by replacing static evaluation with random playouts guided by tree search [36] — but professional strength remained out of reach until deep neural networks were brought to bear.

This survey covers the deep-learning era. We take **AlphaGo** [1] as the starting point and trace the lineage forward through its open reproductions to **KataGo** [13], the strongest open engine and the de facto reference implementation of the modern recipe. Throughout, the unifying object is the *AlphaZero template*: a single neural network with a policy head and a value head, trained on targets distilled from its own tree search during self-play [3]. Nearly every strong engine is a variation on this template; the differences — which are the substance of the field — lie in the search, the network architecture, and the training targets. We organize the survey around exactly those axes, and note along the way the recent work that challenges whether each ingredient (a policy head, MCTS at all) is truly necessary (Section 3).

2 Foundations: the AlphaGo family

Modern Go AI begins with **AlphaGo** [1], which combined deep convolutional policy/value networks with MCTS and bootstrapped from human game records plus self-play, defeating a top professional in 2016. **AlphaGo Zero** [2] removed the human data entirely: a single residual network, trained purely from self-play with MCTS as a policy-improvement operator, surpassed all prior versions.

AlphaZero [3] generalized the same recipe to chess and shogi, establishing the now-standard template: one network with a policy head and a value head, trained on targets distilled from its own MCTS search. A retrospective of the pre-AlphaGo engines and the transition to deep learning is given by the CGI group [36].

3 Search and planning

Before neural networks, game-playing search split along the branching factor. Perfect-information games with modest branching yielded to **minimax** with **alpha–beta** pruning over a handcrafted evaluation — the line that produced Deep Blue's 1997 defeat of Kasparov [50] and that underpinned classical chess engines for two decades (modern Stockfish still searches with alpha–beta but has replaced its handcrafted evaluation with an NNUE network, Section 7). Go's far larger branching factor defeated this approach; the breakthrough that made it tractable was **Monte-Carlo Tree Search**. **UCT** [52] cast move selection as a bandit problem, using an upper-confidence rule to balance exploration against exploitation and growing the tree asymmetrically toward promising lines, and **RAVE** (rapid action-value estimation) sharpened early estimates by sharing statistics across transposed moves [53]. These techniques carried the pre-AlphaGo engines (Crazy Stone, Zen) to strong-amateur strength; Browne et al. survey the MCTS family [54]. AlphaGo's contribution to *search* was to replace hand-built rollout policies and evaluations with learned ones through the **PUCT** rule, which folds a neural policy prior into the UCT selection formula.

At play time, an AlphaZero-style engine does not trust the network's raw policy; it runs MCTS with the PUCT selection rule, using the network's value head to evaluate leaves and its policy head as a prior. The search both improves the move actually played and, during training, produces the visit-count distribution used as the policy target. Several lines of work refine this loop.

MuZero [4] removes the assumption of a known environment model: it learns a latent dynamics model jointly with policy, value, and reward, and plans in that learned model. On Go, chess, and shogi it matched AlphaZero *without* being given the rules, and set the state of the art on Atari. For perfect-information Go the practical effect is modest — the rules are known — but MuZero is the conceptual bridge from "search a known game tree" to "search a learned model." A line of *sample-efficiency* successors makes learned-model planning practical on far less data — **EfficientZero** added a self-supervised consistency loss to reach *superhuman* mean Atari performance from two hours of data (the Atari 100k benchmark) [61], and **EfficientZero V2** generalized the recipe across discrete and continuous control [62]; **ReZero** reuses buffer information via backward-view reanalysis [9], while other work demystifies what MuZero's learned latent model actually represents during planning [8].

Gumbel MuZero / Gumbel AlphaZero [5] rethinks how MCTS selects actions. Standard PUCT can fail to be a sound policy-improvement operator when the simulation budget is small, because not all root actions get visited. The Gumbel variant samples root actions without replacement and allocates simulations by Sequential Halving, guaranteeing policy improvement (when action values are well estimated) and markedly better play at *few* simulations — valuable for cheap engines and for low-latency deployment, where every visit is expensive. **MiniZero** [7] provides a controlled comparison of AlphaZero, MuZero, and their Gumbel variants on 9×9 Go and other games, making the low-simulation regime an empirical rather than anecdotal question.

Recent work continues to refine self-play search control. **Search-contempt** [11] and **regret-guided search control** [10] both target the efficiency of the data-generation search, aiming to learn more per self-play game. Beyond perfect information, **Student of Games** [6] unifies AlphaZero-style search-and-self-play with counterfactual regret minimization, applying one algorithm to both perfect-

and imperfect-information games — evidence that the planning template generalizes well beyond Go.

Two recent results question how essential the template's individual ingredients really are. **Athénan** [63] is competitive with AlphaZero across a range of board games using a *minimax*-based self-play algorithm ("Descent") with *no* policy head (its strongest results are on games smaller than 19×19 Go), and **QZero** [64] reports AlphaGo-level Go from *model-free* reinforcement learning that uses no MCTS during training at all. Neither has displaced KataGo in practice, but together they are a useful reminder that "policy/value network + MCTS" is the dominant recipe, not a proven necessity.

4 Network architectures

Given a fixed search, the network that evaluates positions is the other half of an engine's strength. We trace the trunk from the AlphaZero-era residual stack through KataGo's nested-bottleneck refinement and global pooling, then to the recent wave of transformer and sequence-model backbones.

4.1 Residual trunks and the road to nested bottlenecks

The AlphaZero-era trunk is a stack of *regular* pre-activation residual blocks (two 3×3 convolutions plus a skip connection). The *bottleneck* residual block — a 1×1 convolution that reduces channel width, a 3×3 at the reduced width, and a 1×1 that restores width — comes from ResNet [12] and trades width for depth at fixed cost.

KataGo's **nested bottleneck** blocks (the "nbt" in nets such as `b18c384nbt` and `b40c768nbt`) refine this idea, and their origin is a neat cross-pollination from the search literature. KataGo had earlier found a plain bottleneck block (a single 3×3 at the reduced width) *worse* than a regular block at equal compute: one inner convolution does not repay the cost of the two surrounding 1×1s. The appendix of the Gumbel MuZero paper [5] observed that a *longer* bottleneck — *two* 3×3 convolutions at the reduced width — does pay back that overhead and beats a regular block. KataGo found three or four inner 3×3s slightly better still, and then paired them into their own residual blocks (adding inner skip connections to ease optimization): residual blocks nested inside a bottleneck residual block, hence "nested bottleneck." The payoff is concrete — a `b18c384nbt` runs at roughly the speed of the older `b40c256` trunk (sometimes faster) yet is only marginally weaker per evaluation than the twice-as-heavy `b60c320` [14]. These nets entered KataGo's official run in 2023 and became its strongest [15]. The design is a KataGo engineering contribution documented in its release notes and methods document rather than a separate peer-reviewed ablation; the conceptual ancestor is the ResNet bottleneck [12]. Lightweight convolutional alternatives have also been explored, e.g. MobileNet-style trunks for Go [16].

4.2 Global pooling

A stack of local 3×3 convolutions cannot easily represent global quantities — who is ahead, the total score, the life-and-death status of a large group. KataGo addresses this with *global pooling* residual blocks that mean/max-pool over the whole board and broadcast the result back as a per-channel bias [13]. This gives the trunk a cheap channel for board-wide reasoning and is one reason KataGo's score and ownership heads work as well as they do. The same fully-convolutional-plus-global-pooling design also makes a net board-size agnostic, as exploited by Polygames [66].

4.3 Transformer backbones

The most active recent architectural question is whether attention should replace or augment the convolutional trunk. Convolutions have a fixed local receptive field, whereas many decisive Go relationships — ladders, large-scale capturing races, cyclic groups — are long-range. Several works study this directly, adapting the vision transformer [65] from image recognition to the board. **Vision Transformers for Computer Go** [17] benchmarks ViT-style backbones against ResNet inside an AlphaZero-style policy/value network. **ResTNet** [18] interleaves residual and transformer blocks within the trunk, reporting that the transformer blocks help with exactly the long-range patterns (such as ladders) that pure-CNN trunks handle poorly, and that the hybrid is also less vulnerable to the cyclic adversarial attack discussed in Section 6. **AlphaViT** [19] uses a transformer backbone to build a single agent that plays multiple games and variable board sizes.

The transformer wave is broader than Go. In chess, **ChessBench** [22] trains large transformers by supervised learning on engine-annotated positions and reaches grandmaster-level play without any explicit search, while **Chessformer** [21] trains an encoder-only transformer by supervised learning on a *static* dataset of AlphaZero self-play games (not online self-play). A careful counterpoint, **AlphaVile** [20], finds for chess that improving the input *representation* of an AlphaZero network buys more strength than swapping its convolutions for a transformer — a reminder that architecture is not the only lever. Transformer backbones have also been applied to Chinese chess (Xiangqi) [23].

4.4 Sequence models and emergent world models

A distinct line treats the game as a token stream rather than a board to be convolved over. The **Decision Transformer** [24] casts reinforcement learning as return-conditioned sequence modeling, a framing that many board-game agents build on. **Othello-GPT** [25] showed that a transformer trained only to predict legal Othello moves develops an internal representation of the board state that can be read out and causally edited; follow-up work found this emergent world model to be essentially *linear* [26]. Pushing further, **VideoWorld** [27] learns to play Go from unlabeled video alone — no rules, reward, or search — reaching a reported 5-dan professional level on its Video-GoBench benchmark with a 300M-parameter model, suggesting that planning behavior can emerge from pure sequence prediction (though how far such a benchmark reflects true tournament strength is open). These results matter for Go because they probe what a learned representation actually contains, a question that the robustness literature (Section 6) makes urgent.

5 Engines, reproductions, and the tournament era

The years 2017–2021 saw a crowded field of strong engines competing at the UEC Cup, the ICGA Computer Olympiad, and the World AI Go Open, alongside online rating servers such as CGOS. Surveying the engines is the most direct way to see which ideas mattered in practice. Table 1 summarizes the principal engines of the deep-learning era; we focus on those with the clearest published descriptions or historical weight, and so pass over several competitive programs (e.g. AQ, and the earlier MCTS-era Pachi, Ray, and Aya).

Open reproductions. ELF OpenGo [28] was the first open-source engine to convincingly demonstrate superhuman play (a 20:0 record versus top professionals) and, importantly, published an *analysis* of AlphaZero-style training together with pre-trained models and a large self-play dataset; its predecessor **DarkForest** [29] was an earlier FAIR neural-network Go program from before the self-play era. ELF reached superhuman strength after roughly 2,000 GPUs for nine

Engine	Year	Origin	Open	Notable
AlphaGo	2016	DeepMind	no	First to beat a top professional; CNN + MCTS, human data + self-play [1].
AlphaGo Zero / AlphaZero	2017–18	DeepMind	no	Superhuman from self-play alone; one residual net, the template for all that follow [2, 3].
DarkForest	2016	Facebook (FAIR)	yes	Pre-self-play CNN Go program [29].
ELF OpenGo	2019	Facebook (FAIR)	yes	First open superhuman engine (20:0 vs. pros); ∼2,000 GPUs, ∼2 weeks [28].
Leela Zero	2017–	community	yes	Distributed reproduction to pro strength; no paper [31].
Minigo	2018	Google	yes	Reproducibility case study on TPUs [30].
SAI	2019	academic (IT)	yes	Komi-parameterized value for handicap/score play [32].
CGI	2017–	NYCU (Taiwan)	part.	Multi-labelled value nets; population-based training [34, 35].
PhoenixGo	2018	Tencent	yes	Won the 2018 World AI Go Tournament [33].
Fine Art (*Jueyi*)	2017–	Tencent	no	Top closed industrial engine; no system paper.
Golaxy	2018–	Beijing Shenke	no	Top closed industrial engine; no system paper.
KataGo	2019–	D. Wu / open	yes	Strongest open engine; ∼50× training-efficiency gain; nbt trunk [13].

Table 1: Principal deep-learning-era Go engines. "Open" indicates whether code/weights are publicly available. Dashes elsewhere reflect engines with no published system paper.

days (its full run spanned about two weeks) — a cost that motivated the efficiency work below. **Leela Zero** [31] was a distributed, community-run reimplementation that reached professional strength; it has no formal paper but was hugely influential as open infrastructure. **Minigo** [30] is a Google reproduction notable for documenting the reproducibility challenges of the recipe. **SAI** [32] extended the self-play framework with a komi-parameterized value function to play under handicap and target score margins.

Industrial and academic engines. Tencent fielded two related engines: the open-source **PhoenixGo** [33], which won the 2018 World AI Go Tournament, and the closed-source **Fine Art** (*Jueyi*), among the strongest engines of the era; **Golaxy** was another top industrial engine. Neither Fine Art nor Golaxy has a peer-reviewed system paper, which is itself a notable feature of the landscape. On the academic side, the CGI group produced a sustained line of work: **multi-labelled value networks** that predict win rates across different komi values and add a board-ownership evaluation [34] — ideas that anticipate KataGo's komi-aware value and ownership targets — and the use of **population-based training** to tune AlphaZero hyperparameters during self-play [35].

KataGo: efficiency and richer targets. **KataGo** [13] is the strongest open engine and the de facto reference for the modern recipe. Its headline result is a ∼50× reduction in training compute relative to comparable AlphaZero reproductions: it surpassed ELF's final model after 19 days on fewer than 30 GPUs. The gains come from a set of mostly orthogonal techniques:

- **Auxiliary targets** — in addition to policy and game-result value, the net predicts *board*

ownership and final *score* (both a distribution and a lead). These dense, spatially-grounded targets accelerate representation learning and give the engine a notion of margin, not just win probability.
- **Global pooling** (Section 4) for board-wide reasoning.
- **Playout-cap randomization** — most self-play moves use a small search; a randomized minority use a large one, decoupling the cheap data-generation policy from the expensive high-quality value/policy targets.
- **Policy-target pruning** and other self-play/target refinements that reduce noise in the distilled targets.

KataGo is the canonical reference for everything at the training-target and search level.

Efficiency as a research target. The cost of self-play has become a topic in its own right. Empirical *scaling laws* for board games relate compute to playing strength under the AlphaZero paradigm [37]; further work expedites self-play learning [38] and diversifies the initial states used for self-play to improve sample efficiency [39]. Knowledge *distillation* from a strong teacher into a small network — demonstrated for Gomoku in **Rapfi** [40] — is a complementary route to cheap, deployable engines. A broad survey of self-play methods places this work in the wider reinforcement-learning context [41].

6 The adversarial-robustness turn

The most striking recent result is that "superhuman" Go AIs are *not* robust. **Wang et al.** [42] trained adversarial policies against KataGo and won >97% of games even against KataGo at superhuman search settings. The exploit is the now-famous *cyclic* attack: the victim network misjudges the life-and-death status of a large group whose stones enclose the opponent in a ring-like (cyclic) shape, and that group is then captured wholesale. The attack transfers zero-shot to other top engines (Leela Zero, ELF) and, tellingly, is comprehensible enough that human *experts* can reproduce it by hand against superhuman bots — even though the adversarial policy itself is so narrow that ordinary amateur players beat it easily. The methodology descends from earlier work showing that adversarial policies exist in zero-sum games generally [44], and a parallel study examined the robustness of AlphaZero-like agents to adversarial board perturbations [45].

The follow-up, **"Can Go AIs be adversarially robust?"** [43], tested three natural defenses — adversarial training on hand-built positions, iterated adversarial training, and architectural changes (including a vision transformer). Each blunts known attacks but none survives a freshly trained adversary; a new attacker still beats KataGo's hardened end-of-2023 network roughly two thirds of the time, almost always via some realization of the same cyclic motif. The lesson is sobering and general: extreme average-case superhuman strength does not imply worst-case robustness, even in a closed, fully-observable game. One constructive response is to attack the underlying weakness directly — formal life-and-death solvers that can certify the status of the very groups the cyclic attack exploits [46].

7 Beyond Go: the broader game-AI context

The AlphaZero template did not arise in isolation, and Go is one point on a wider arc.

- **Self-play RL** long predates AlphaGo: **TD-Gammon** reached expert backgammon strength by temporal-difference learning from self-play in the 1990s [51], the original demonstration

that a network could bootstrap its own training signal.

- In **chess**, the classical alpha–beta engine (Deep Blue [50]) was eventually matched and overtaken by hybrid search-plus-network systems: Stockfish replaced its handcrafted evaluation with a small, efficiently-updatable network (NNUE), while Leela Chess Zero followed the AlphaZero self-play recipe directly. Much of the architecture experimentation most relevant to Go now happens in chess — transformer policies trained by supervised learning [22] or self-play [21], the representation-over-architecture finding of AlphaVile [20], and graph-neural-network board encodings that transfer across board sizes (AlphaGateau) [60].
- The **Atari** benchmark drove model-free deep RL — DQN [55], then actor-critic and policy-gradient methods such as A3C [56] and PPO [57] — and remained a model-free stronghold until MuZero [4] brought learned-model planning to parity.
- **Imperfect-information and real-time** games are where the template is still being stretched: OpenAI Five reached professional Dota 2 [59] and AlphaStar reached Grandmaster StarCraft II [58], both through large-scale self-play, while Student of Games (Section 3) unifies the perfect- and imperfect-information cases [6].

These domains share Go's core ingredients — self-play, a policy/value network, and search or planning — and differ mainly in how much of the environment must be *learned* and how much hidden information the agent must reason about.

8 Emerging directions

Four threads are worth watching. First, **language models as planners**: recent work uses an LLM as the policy/value function for external MCTS, or has it generate a linearized search internally, reaching strong play across several board games at human-like search budgets [47], and related work re-injects human conceptual knowledge into Go engines that the pure-self-play recipe had discarded [48]. Second, **interpretability**: probing studies find evidence of learned look-ahead inside chess-playing networks [49], and Go-native work extracts human-teachable interaction patterns from KataGo's value network [67], complementing the Othello world-model results and bearing directly on *why* the cyclic blind spot exists. Third, **architecture search** continues, with the transformer-vs-convolution question (Section 4) far from settled — the strongest current evidence is that hybrids help on long-range patterns [18] while representation can matter more than the backbone [20]. Finally, the human side: trained on millions of professional moves, superhuman engines have been shown to *expand* human play toward novel strategies [68], turning the engines into a tool for human discovery rather than only a benchmark to chase.

9 Outlook

The modern Go engine is a remarkably stable design: an AlphaZero-template network with KataGo's auxiliary targets and global pooling, a residual or nested-bottleneck trunk, and MCTS (increasingly Gumbel-style at low budgets) as both a play-time and a training-time operator. The open questions are no longer about reaching superhuman strength — that is solved and cheaply reproducible — but about its limits: how to split a fixed parameter budget between depth and width, whether attention earns its place in the trunk, how few visits modern search can get away with, and, most pointedly, how to make an engine that is strong *and* robust. The cyclic-attack literature suggests these last two goals are in tension: self-play concentrates training data on the high-quality lines strong engines actually reach, so positions like the cyclic shape — which a competent self-play opponent would

never enter — remain effectively out of distribution no matter how strong average play becomes, and an adversary trained explicitly to seek them out keeps finding a fresh exploit. Whether better coverage (adversarial or curriculum self-play), a longer-range architecture (Section 4), or explicit certification (life-and-death solvers) is the right fix remains the field's most interesting open problem.

References

[1] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature* **529**, 484–489 (2016). https://doi.org/10.1038/nature16961

[2] D. Silver *et al.*, "Mastering the game of Go without human knowledge," *Nature* **550**, 354–359 (2017). https://doi.org/10.1038/nature24270

[3] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science* **362**, 1140–1144 (2018). https://doi.org/10.1126/science.aar6404

[4] J. Schrittwieser *et al.*, "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature* **588**, 604–609 (2020). arXiv:1911.08265. https://arxiv.org/abs/1911.08265

[5] I. Danihelka, A. Guez, J. Schrittwieser, D. Silver, "Policy improvement by planning with Gumbel," *ICLR*, 2022. https://openreview.net/forum?id=bERaNdoegnO

[6] M. Schmid *et al.*, "Student of Games: A unified learning algorithm for both perfect and imperfect information games," *Science Advances* **9**, 2023. arXiv:2112.03178. https://arxiv.org/abs/2112.03178

[7] T. Wu *et al.*, "MiniZero: Comparative analysis of AlphaZero and MuZero on Go, Othello, and Atari games," *IEEE Transactions on Games*, 2025. arXiv:2310.11305. https://arxiv.org/abs/2310.11305

[8] H. Guei *et al.*, "Demystifying MuZero planning: Interpreting the learned model," 2024. arXiv:2411.04580. https://arxiv.org/abs/2411.04580

[9] C. Xuan *et al.*, "ReZero: Boosting MCTS-based algorithms by backward-view and entire-buffer reanalyze," 2024. arXiv:2404.16364. https://arxiv.org/abs/2404.16364

[10] Y.-J. Tsai *et al.*, "Regret-guided search control for efficient learning in AlphaZero," 2026. arXiv:2602.20809. https://arxiv.org/abs/2602.20809

[11] S. Joshi, "Search-contempt: A hybrid MCTS algorithm for training AlphaZero-like engines with better computational efficiency," 2025. arXiv:2504.07757. https://arxiv.org/abs/2504.07757

[12] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. arXiv:1512.03385. https://arxiv.org/abs/1512.03385

[13] D. J. Wu, "Accelerating Self-Play Learning in Go," 2019. arXiv:1902.10565. https://arxiv.org/abs/1902.10565

[14] D. J. Wu and KataGo contributors, "KataGo Methods" (docs/KataGoMethods.md), lightvector/KataGo. https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md

[15] lightvector/KataGo, "Release v1.12.0: New Neural Net Architecture" (nested bottleneck nets), 2023. https://github.com/lightvector/KataGo/releases/tag/v1.12.0

[16] T. Cazenave, "Mobile Networks for Computer Go," 2020. arXiv:2008.10080. https://arxiv.org/abs/2008.10080

[17] A. Sagri, T. Cazenave, J. Arjonilla, A. Saffidine, "Vision Transformers for Computer Go," 2023. arXiv:2309.12675. https://arxiv.org/abs/2309.12675

[18] Y. Ju, T.-R. Wu, C.-C. Shih, I.-C. Wu, "Bridging Local and Global Knowledge via Transformer in Board Games," *IJCAI*, 2025. arXiv:2410.05347. https://arxiv.org/abs/2410.05347

[19] K. Fujita, "AlphaViT: A flexible game-playing AI for multiple games and variable board sizes," 2024. arXiv:2408.13871. https://arxiv.org/abs/2408.13871

[20] J. Czech, J. Blüml, K. Kersting, H. Steingrímsson, "Representation matters for mastering chess: Improved feature representation in AlphaZero outperforms switching to transformers," 2024. arXiv:2304.14918. https://arxiv.org/abs/2304.14918

[21] D. Monroe and P. A. Chalmers, "Mastering Chess with a Transformer Model," 2024. arXiv:2409.12272. https://arxiv.org/abs/2409.12272

[22] A. Ruoss *et al.*, "Amortized Planning with Large-Scale Transformers: A Case Study on Chess," *NeurIPS*, 2024. arXiv:2402.04494. https://arxiv.org/abs/2402.04494

[23] Y. Chen, Z. Lin, P. Shu, "Mastering Chinese Chess AI (Xiangqi) Without Search," 2024. arXiv:2410.04865. https://arxiv.org/abs/2410.04865

[24] L. Chen *et al.*, "Decision Transformer: Reinforcement learning via sequence modeling," *NeurIPS*, 2021. arXiv:2106.01345. https://arxiv.org/abs/2106.01345

[25] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, M. Wattenberg, "Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task," *ICLR*, 2023. arXiv:2210.13382. https://arxiv.org/abs/2210.13382

[26] N. Nanda, A. Lee, M. Wattenberg, "Emergent Linear Representations in World Models of Self-Supervised Sequence Models," 2023. arXiv:2309.00941. https://arxiv.org/abs/2309.00941

[27] Z. Ren *et al.*, "VideoWorld: Exploring Knowledge Learning from Unlabeled Videos," *CVPR*, 2025. arXiv:2501.09781. https://arxiv.org/abs/2501.09781

[28] Y. Tian *et al.*, "ELF OpenGo: An Analysis and Open Reimplementation of AlphaZero," *ICML* (PMLR 97), 2019. arXiv:1902.04522. https://arxiv.org/abs/1902.04522

[29] Y. Tian and Y. Zhu, "Better Computer Go Player with Neural Network and Long-term Prediction," *ICLR*, 2016. arXiv:1511.06410. https://arxiv.org/abs/1511.06410

[30] B. Lee, A. Jackson, T. Madams, S. Troisi, D. Jones, "Minigo: A Case Study in Reproducing Reinforcement Learning Research," *ICLR 2019 Workshop (RML)*. https://openreview.net/forum?id=H1eerhIpLV

[31] G.-C. Pascutto, "Leela Zero" (open-source AlphaGo Zero reimplementation), 2017–. https://github.com/leela-zero/leela-zero

[32] F. Morandin, G. Amato, R. Gini, C. Metta, M. Parton, G.-C. Pascutto, "SAI, a Sensible Artificial Intelligence that plays Go," *IJCNN*, 2019. arXiv:1809.03928. https://arxiv.org/abs/1809.03928

[33] Tencent, "PhoenixGo" (open-source AlphaGo Zero implementation; sibling of Fine Art / Jueyi), 2018. https://github.com/Tencent/PhoenixGo

[34] T.-R. Wu *et al.*, "Multi-Labelled Value Networks for Computer Go," *IEEE Transactions on Games* **10**(4), 2018. arXiv:1705.10701. https://arxiv.org/abs/1705.10701

[35] T.-R. Wu, T.-H. Wei, I.-C. Wu, "Accelerating and Improving AlphaZero Using Population Based Training," *AAAI*, 2020. arXiv:2003.06212. https://arxiv.org/abs/2003.06212

[36] C.-S. Lee, M.-H. Wang, S.-J. Yen, T.-H. Wei, I.-C. Wu *et al.*, "Human vs. Computer Go: Review and Prospect," *IEEE Computational Intelligence Magazine* **11**(3), 2016. arXiv:1606.02032. https://arxiv.org/abs/1606.02032

[37] A. L. Jones, "Scaling Scaling Laws with Board Games," 2021. arXiv:2104.03113. https://arxiv.org/abs/2104.03113

[38] Y. Björnsson *et al.*, "Expediting Self-Play Learning in AlphaZero-Style Game-Playing Agents," *ECAI*, 2023. https://doi.org/10.3233/FAIA230279

[39] K. Demura and T. Kaneko, "Initial State Diversification for Efficient AlphaZero-Style Training," *ICGA Journal*, 2024. https://doi.org/10.3233/ICG-240255

[40] H. Jin *et al.*, "Rapfi: Distilling Efficient Neural Network for the Game of Gomoku," 2025. arXiv:2503.13178. https://arxiv.org/abs/2503.13178

[41] R. Zhang *et al.*, "A Survey on Self-Play Methods in Reinforcement Learning," 2025. arXiv:2408.01072. https://arxiv.org/abs/2408.01072

[42] T. T. Wang *et al.*, "Adversarial Policies Beat Superhuman Go AIs," *ICML* (PMLR 202), 2023. arXiv:2211.00241. https://arxiv.org/abs/2211.00241

[43] T. Tseng, E. McLean, K. Pelrine, T. T. Wang, A. Gleave, "Can Go AIs be adversarially robust?," 2024. arXiv:2406.12843. https://arxiv.org/abs/2406.12843

[44] A. Gleave *et al.*, "Adversarial Policies: Attacking Deep Reinforcement Learning," *ICLR*, 2020. arXiv:1905.10615. https://arxiv.org/abs/1905.10615

[45] L.-C. Lan *et al.*, "Are AlphaZero-like Agents Robust to Adversarial Perturbations?," *NeurIPS*, 2022. arXiv:2211.03769. https://arxiv.org/abs/2211.03769

[46] C.-C. Shih *et al.*, "A Study of Solving Life-and-Death Problems in Go Using Relevance-Zone-Based Solvers," *IEEE Transactions on Games*, 2026. arXiv:2512.21365. https://arxiv.org/abs/2512.21365

[47] J. Schultz *et al.*, "Mastering Board Games by External and Internal Planning with Language Models," 2025. arXiv:2412.12119. https://arxiv.org/abs/2412.12119

[48] Y. Ma *et al.*, "Mixing Expert Knowledge: Bring Human Thoughts Back to the Game of Go," 2026. arXiv:2601.16447. https://arxiv.org/abs/2601.16447

[49] E. Jenner *et al.*, "Evidence of Learned Look-Ahead in a Chess-Playing Neural Network," 2024. arXiv:2406.00877. https://arxiv.org/abs/2406.00877

[50] M. Campbell, A. J. Hoane Jr., F.-h. Hsu, "Deep Blue," *Artificial Intelligence* **134**(1–2), 57–83 (2002). https://doi.org/10.1016/S0004-3702(01)00129-1

[51] G. Tesauro, "Temporal Difference Learning and TD-Gammon," *Communications of the ACM* **38**(3), 58–68 (1995). https://doi.org/10.1145/203330.203343

[52] L. Kocsis and C. Szepesvári, "Bandit Based Monte-Carlo Planning," *ECML*, 2006. https://doi.org/10.1007/11871842_29

[53] S. Gelly and D. Silver, "Combining Online and Offline Knowledge in UCT," *ICML*, 2007. https://doi.org/10.1145/1273496.1273531

[54] C. Browne *et al.*, "A Survey of Monte Carlo Tree Search Methods," *IEEE Transactions on Computational Intelligence and AI in Games* **4**(1), 1–43 (2012). https://doi.org/10.1109/TCIAIG.2012.2186810

[55] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature* **518**, 529–533 (2015). https://doi.org/10.1038/nature14236

[56] V. Mnih *et al.*, "Asynchronous Methods for Deep Reinforcement Learning," *ICML*, 2016. arXiv:1602.01783. https://arxiv.org/abs/1602.01783

[57] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms," 2017. arXiv:1707.06347. https://arxiv.org/abs/1707.06347

[58] O. Vinyals *et al.*, "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature* **575**, 350–354 (2019). https://doi.org/10.1038/s41586-019-1724-z

[59] C. Berner *et al.* (OpenAI), "Dota 2 with Large Scale Deep Reinforcement Learning," 2019. arXiv:1912.06680. https://arxiv.org/abs/1912.06680

[60] T. Rigaux and H. Kashima, "Enhancing Chess Reinforcement Learning with Graph Representation" (AlphaGateau), *NeurIPS*, 2024. arXiv:2410.23753. https://arxiv.org/abs/2410.23753

[61] W. Ye, S. Liu, T. Kurutach, P. Abbeel, Y. Gao, "Mastering Atari Games with Limited Data" (EfficientZero), *NeurIPS*, 2021. arXiv:2111.00210. https://arxiv.org/abs/2111.00210

[62] S. Wang *et al.*, "EfficientZero V2: Mastering Discrete and Continuous Control with Limited Data," *ICML*, 2024. arXiv:2403.00564. https://arxiv.org/abs/2403.00564

[63] Q. Cohen-Solal and T. Cazenave, "Minimax Strikes Back" (Athénan / Descent self-play), 2023. arXiv:2012.10700. https://arxiv.org/abs/2012.10700

[64] Z. Liu and J. Wang, "Mastering the Game of Go with Self-play Experience Replay" (QZero), 2026. arXiv:2601.03306. https://arxiv.org/abs/2601.03306

[65] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale" (ViT), *ICLR*, 2021. arXiv:2010.11929. https://arxiv.org/abs/2010.11929

[66] T. Cazenave *et al.*, "Polygames: Improved Zero Learning," *ICGA Journal*, 2020. arXiv:2001.09832. https://arxiv.org/abs/2001.09832

[67] H. Zhou *et al.*, "Explaining How a Neural Network Plays the Go Game," 2023. arXiv:2310.09838. https://arxiv.org/abs/2310.09838

[68] M. Shin, J. Kim, B. van Opheusden, T. L. Griffiths, "Superhuman Artificial Intelligence Can Improve Human Decision Making by Increasing Novelty," *PNAS* **120**(12), 2023. https://doi.org/10.1073/pnas.2214840120